

UNIT II

- Disjoint set operations.
- union and find algorithms.
- Spanning trees.
- connected and bi-connected components.

Spanning trees and connected components:-

A spanning tree for a connected, undirected graph $G = (V, E)$ is a subgraph of G that is an undirected tree and containing all the vertices of G .

⇒ We can also define a spanning tree as 'A subgraph $T = (V, E)$ of G is a spanning tree of G iff T is a tree.

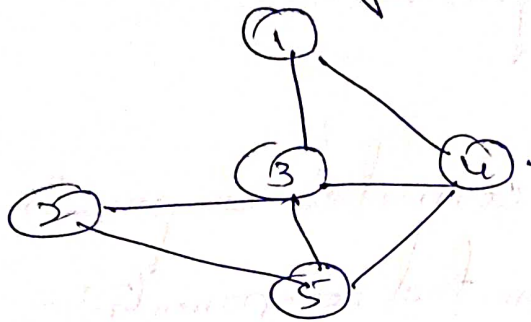
⇒ A spanning tree of a graph should include all the vertices and a subset of edges (E').

⇒ The BFS and the BFT algorithms are applied to test whether a graph G is connected or not and to obtain the connected components of G .

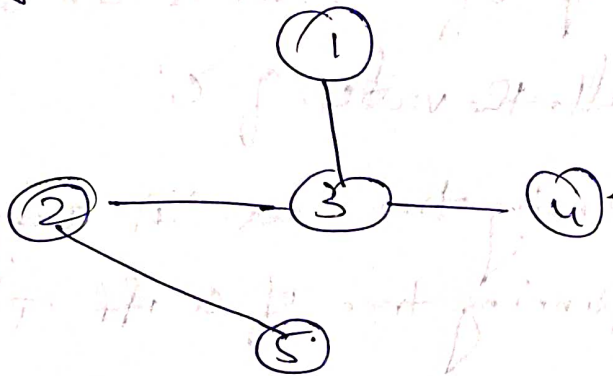
⇒ Consider the set of all edges (u, w) , where all vertices w are adjacent to u and are not visited. Refer to the BFS algorithm. It is established that this set of edges give the spanning tree of G . If G is

connected we obtain depth first search spanning trees similarly.

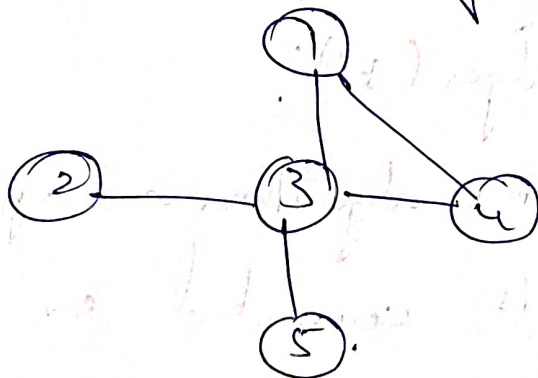
⇒ Consider the following graph.



⇒ The following are the DFS and BFS spanning trees of the graph.



DFS Spanning Tree.



BFS spanning Tree.

Biconnected components:-

A connected undirected graph G is said to be biconnected if it remains connected after removal of any one vertex and the edges that are incident upon the vertex.

A biconnected component (bicomponent) of an undirected graph is a maximal biconnected subgraph. That is, a biconnected subgraph not contained in any larger biconnected subgraph.

- Articulation point (or) cut point
- Construction of Biconnected graph.
- Depth-first spanning Tree.
- ~~Algorithm for DFS.~~
- Algorithm for Biconnected components.

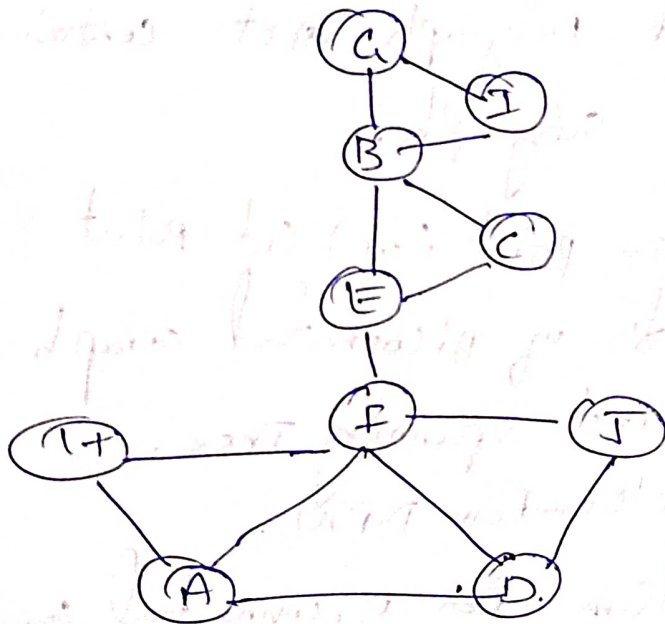
Articulation point (or) cut point:

A vertex v is an articulation point (or) cut point for an undirected graph G if there are distinct vertices w and x , distinct from v also such that v is in every path from w to x .

The definition of articulation point (or) cut point can be once again defined using simple terms i.e. A vertex v in a connected graph is an cut point if and only if the deletion of vertex v together with all edges incident to v disconnects the graph into two (or) more non empty components.

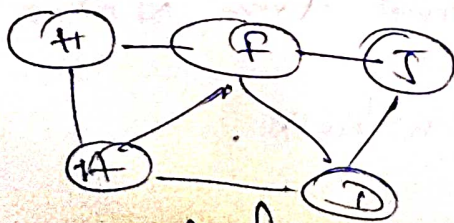
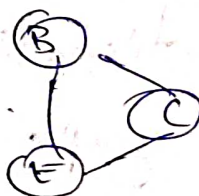
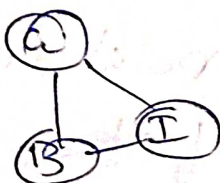
example:-

(1)



undirected graph.

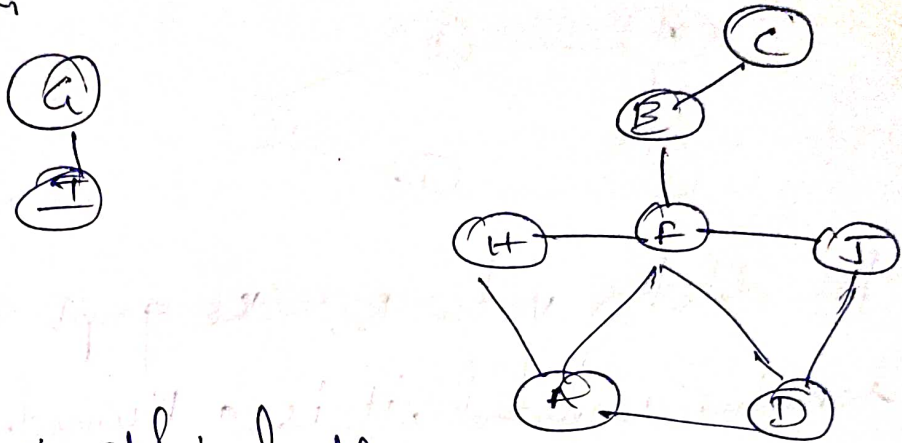
(2)



Disconnected components of graph 1

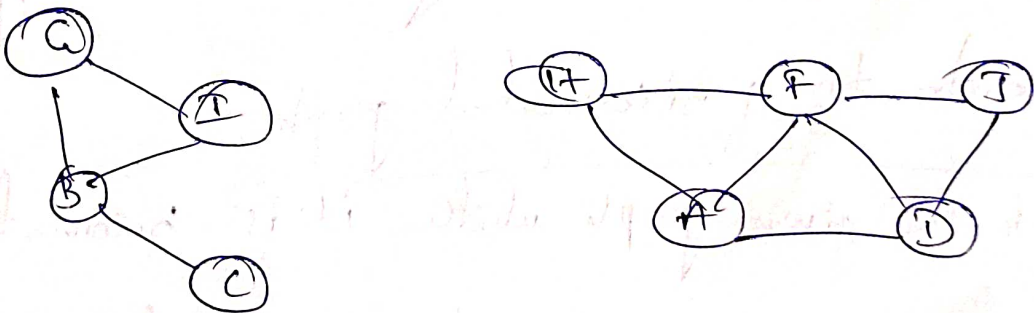
3. Articulation point:-

- (i) After deleting vertex B and incident edges of B, the given graph is divided into two nonempty components



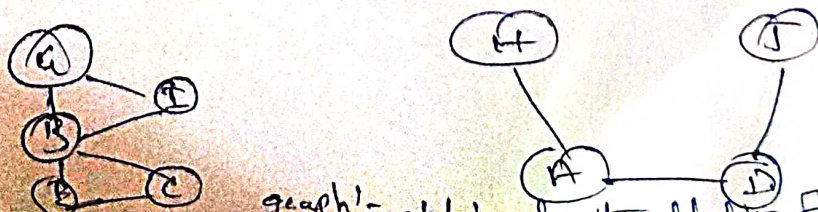
graph obtained after deleting B from graph 1.

- (ii) After deleting the vertex E and incident edges of E the resulting nonempty components are.



graph:- obtained after deleting E from graph 1.

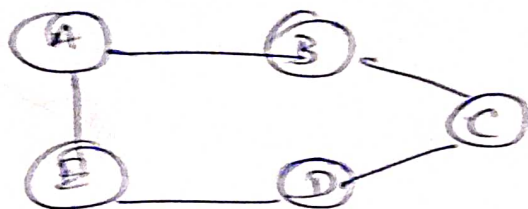
- (iii) After deleting vertex F and incident edges of F, the given graph is divided into 2 nonempty components



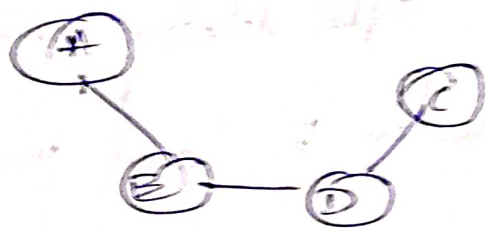
graph:- obtained after deleting F from graph 1 (2)

→ from the above points (i, ii and iii) it is clear that the articulation points of given graph are B, E and F.

④ Biconnected graph



After deleting vertex B, the graph structure still remains connected, it is a biconnected graph.

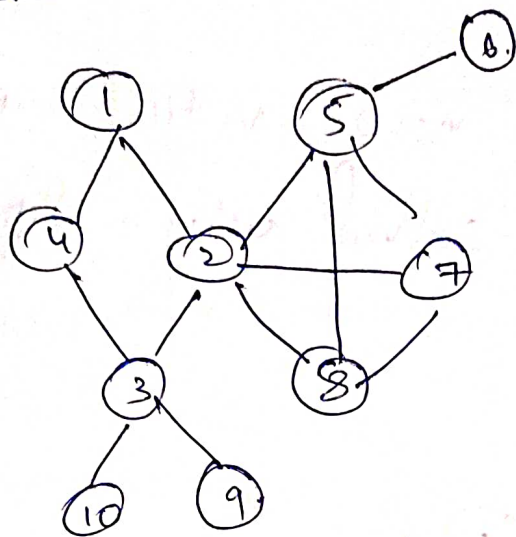


graph after deleting B from above graph 2.

Construction of Biconnected graph:-

- 1.) check the given graph whether it is biconnected (or) not.
- 2.) If the given graph is not biconnected then identify all the articulation points.
- 3.) If articulation point exist, determine a set of edges whose inclusion makes the graph biconnected.

Example:-



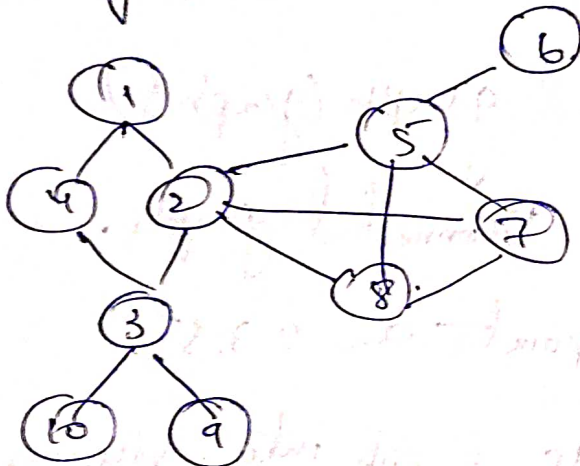
An undirected graph (graph 3)

- given graph 3 is not a biconnected graph.
- The articulation points are 2, 3, 5
- To Transform the graph into biconnected graph the new edges are included corresponding to the articulation point.
- Edges corresponding to the articulation point
3 - (4, 10), (10, 9)
- edges corresponding to the articulation point
2 - (1, 5), (3, 8)
- Edges correspond to the articulation point 5 - (6, 7)

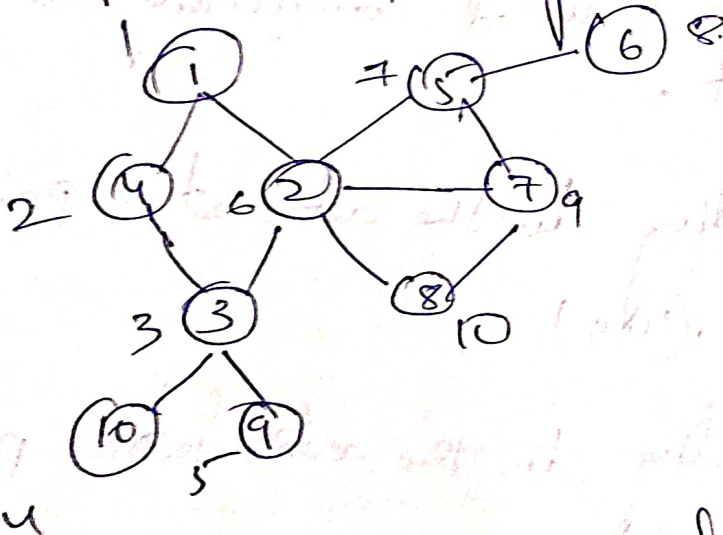
Problem of identifying the Articulation points and Biconnected components

- If the given graph G has $n \geq 2$ vertices, the Problem is efficiently solved using a depth-first spanning tree of G .

(1) Given graph.



(2) Depth-first spanning tree is



In the fig there is a no. of outside each vertex corresponds to the order in which a depth-first search visits these vertices and are named as depth-first numbers of the vertex (i.e)

$$dfn[1] = 1$$

$$dfn[4] = 2$$

$$dfn[3] = 3$$

$$dfn[10] = 4$$

$$dfn[9] = 5$$

$$dfn[2] = 6$$

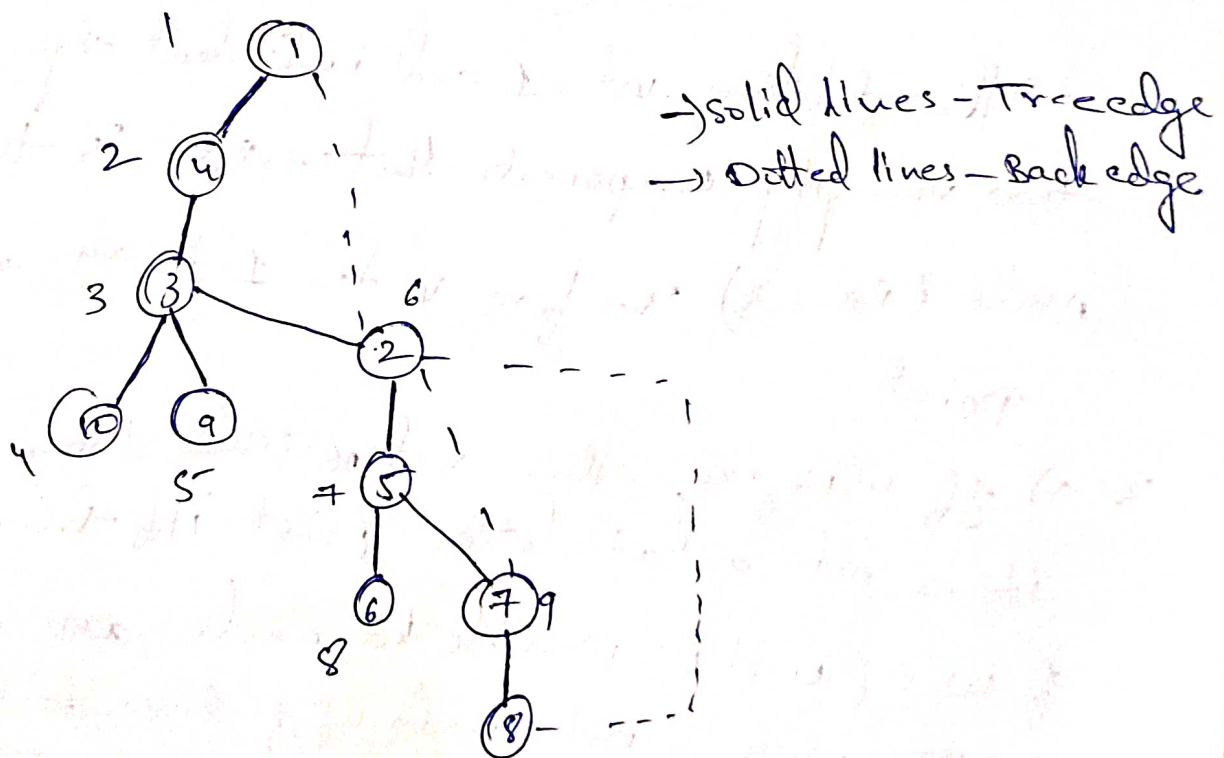
$$dfn[5] = 7$$

$$dfn[6] = 8$$

$$dfn[7] = 9$$

$$dfn[8] = 10$$

The solid edges of figure will form a depth-first spanning tree representation.
is



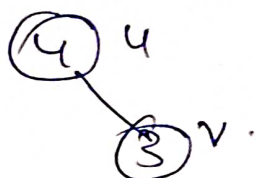
Depthfirst spanning tree

- DFST are very useful in identifying articulation point and biconnected components.

Depth First Spanning Tree properties

1.) If (u, v) is any tree edge of depth first spanning tree G , then either u is an ancestor of v or v is an ancestor of u .

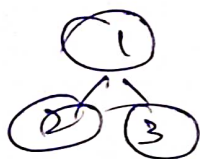
Example:-



A Tree with (u, v) edge.

② The root node of a depth first spanning tree is an articulation point iff it has atleast two children.

Example:-



After deleting vertex 1 and incident edges of the same two nonempty components that exist is two children node (i.e., 2, 3) Therefore vertex 1 is an articulation point.

3 a) If u is any other vertex other than root. then it is not an articulation point iff from every child w of u it is possible to reach an ancestor of u using only a path made up of descending w and a back edge.

note that if this cannot be done for some child w of u then the deletion of vertex u leaves behind at least two nonempty components (one containing the root and the other containing vertex w). This observation leads to a simple rule to identify the articulation points. for each vertex u , define $L(u)$ as follows:

$$L(u) = \min \{ \text{dfs}[u], \min \{ L[w] \mid w \text{ is child of } u \}, \min \{ \text{dfs}[w] \mid (u, w) \text{ is a back edge} \} \}$$

where $\text{dfs}[u]$ is the lowest depth first number that can be reached from u using a path of descendants followed by at most one back edge.

b) u is an articulation point (not a root) iff u has a child w such that $L[w] \geq \text{dfs}[u]$.

example! - For the spanning tree of fig 2 values are computed easily by applying postorder traversal in the depth first spanning tree.

$$L[10] = \min \{ 4, -, - \}$$

$$L[10] = 4$$

$$L[9] = \min \{ 5, -, - \}$$

$$L[9] = 5$$

$$L[6] = \min \{ 8, -, - \}$$

$$L[6] = 8$$

$$L[8] = \min\{10, -, 6\}$$

$$L[8] = 6$$

$$L[7] = \min\{1, 6, 6\}$$

$$L[7] = 1$$

$$L[5] = \min\{7, 6, -\}$$

$$L[5] = 6$$

$$L[2] = \min\{6, 6, 1\}$$

$$L[2] = 1$$

$$L[3] = \min\{3, 1, -\}$$

$$L[3] = 1$$

$$L[4] = \min\{2, 1, -\}$$

$$L[4] = 1$$

$$L[1] = \min\{1, 1, -\}$$

$$L[1] = 1$$

$$L[1:10] = \{1, 1, 1, 1, 6, 8, 6, 6, 5, 4\}$$

→ for the spanning tree of fig. the articulation points are identified using the condition $L[u] > \text{dfn}[v]$ iff u has a child v and u is not the root.

(i) for vertex ② $L[5] > \text{dfn}[2]$

i.e. $6 > 1 \Rightarrow \text{true} \Rightarrow \text{cut point}$

(ii) for vertex ③ $L[10] > \text{dfn}[3]$

i.e. $4 > 3 \Rightarrow \text{true} \Rightarrow \text{cut point}$

(iii) For vertex (4) $L[3] \geq \text{dfn}[4]$

i.e. $1 \geq 2 \Rightarrow \text{false}$.

(iv) For vertex (5) $L[6] \geq \text{dfn}[5]$

i.e. $8 \geq 7 \Rightarrow \text{True} \Rightarrow \text{cut point}$

(v) For vertex (9) $L[8] \geq \text{dfn}[7]$

i.e. $6 \geq 9 \Rightarrow \text{false}$.

The condition to check articulation point is not done for the vertices 6, 8, 9 and 10, since w does not exist in the spanning tree structure and the condition is true for the vertices 2, 3 and 5. Therefore the articulation points are 2, 3 and 5.

Algorithm to find DFN's and L values.

Algorithm Art(u, v)

// u is a start vertex for depth first search, v is the parent if any in the DFST. It is assumed that the global array dfn is initialized to zero and that the global variable num is initialized to 1. n is the root of vertices in G .

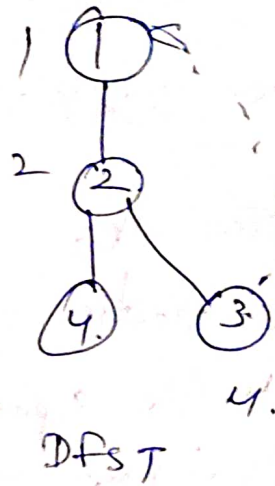
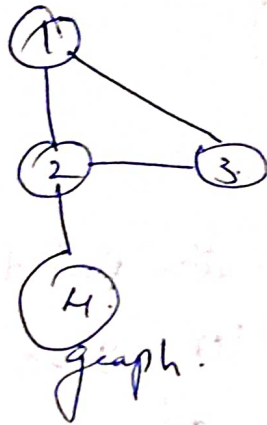
```
1  $\text{dfn}[u] = \text{num}; L[u] = \text{num}; \text{num} = \text{num} + 1;$   
2 For each vertex  $w$  adjacent from  $u$  do  
3   if ( $\text{dfn}[w] = 0$ ) then  
4     Art( $w, u$ );  
5      $L[u] = \min(L[u], L[w])$ 
```

else if $(u \neq v)$ then $L[u] = \min(L[u], dfn[v])$

}

}

Execution of Art (u, v)



(i) Art $(1, 0)$

$dfn[1] = 1$

$L[1] = 1$

$num = 2$

if $(dfn[2] = 0) \leftarrow \text{true}$

Art $(2, 1)$

$L[2] = \min(L[2], L[1])$

$L[2] = \min(1, 1)$

$L[1] = 1$

(ii) Art $(2, 1)$

$dfn[2] = 2$

$L[2] = 2$

$num = 3$

if $(dfn[4] = 0) \leftarrow \text{true}$

Art $(4, 2)$

$$L[u] = \min(L[u], L[w]) = \min(2, 1)$$

$$L[u] = 1$$

(iii) $\text{Art}(u, 2)$

$$\text{dfn}[u] = 3$$

$$L[u] = 3$$

num =

Adjacent vertices for u (i.e) from vertex u is not exist, control returns to $\text{Art}(2, 1)$

$$\text{if}(\text{dfn}[3] = 0) \leftarrow \text{true}$$

~~$\text{Art}(3, 2)$~~

$$\text{dfn}[3] = 4$$

$$L[3] = 4$$

num = 5

$$\text{if}(\text{dfn}[1] = 0) \leftarrow \text{false}$$

$$L[u] = \min(L[u], \text{dfn}[w])$$

$$L[3] = \min(4, 1)$$

$$L[3] = 1$$

computed dfn's and L values are.

$$\text{dfn}[1] = 1 \quad L[1] = 1$$

$$\text{dfn}[2] = 2 \quad L[2] = 1$$

$$\text{dfn}[4] = 3 \quad L[3] = 1$$

$$\text{dfn}[3] = 4 \quad L[u] = 3$$

note:- The computed L values using Art algorithm can be checked with the condition equation.

$$L[u] = \min\{dfn[u], \min\{L[w] \mid w \text{ is a child of } u\}, \min\{L[w] \mid (u, w) \text{ is a back edge}\}\}$$

$$L[u] = \min\{3, -, -\}$$

$$L[u] = 3$$

$$L[3] = \min\{4, -, -\}$$

$$L[3] = 1$$

$$L[2] = \min\{2, 1, -\}$$

$$L[2] = 1$$

$$L[1] = \min\{1, 1, -\}$$

$$L[1] = 1$$

i.e

$$L[1] = 1$$

$$L[2] = 1$$

$$L[3] = 1$$

$$L[u] = 3$$

using the L values the articulation points are computed when $L[w] \geq dfn[u]$ is satisfied.

(i) for vertex ②, $L[u] \geq dfn[2]$
 $3 \geq 2 \Rightarrow \text{true} \Rightarrow \text{cut point}$

The articulation point of the given graph is vertex only.

Analysis: Once $L[u, n]$ has been computed the articulation points can be identified in $O(e)$ time.

→ Therefore the complexity of Art algorithm is $O(n+e)$, where e is the no. of edges and n is the no. of vertices in G .

Algorithm to find Biconnected components

Algorithm Biconp(u, v)

u is a start vertex for depth first search. v is the parent of u in the depth first spanning tree. It is assumed that the global array dfn is initially zero and that the global variable num is initialised to 1. n is the no. of vertices in G .

1 $dfn[u] = num; L[u] = num; num = num + 1;$

for each vertex w adjacent from u do

2 if $(v \neq w)$ and $(dfn[w] < dfn[u])$ then

add (u, w) to the top of a stack s ;

if $(dfn[w] = 0)$ then

2 if $(L[w] \geq dfn[u])$ then

2 write ("new bicomponent")

2 repeat

Delete an edge from the top of the stack s ;

Let this edge be (x, y) ;

write (x, y) ;

} until $((x, y) = (u, w) \text{ or } ((x, y) = (w, u)))$;

}

BiComp(w, u); // w is unvisited.

$LT[u] = \min(LT[u], LT[w])$

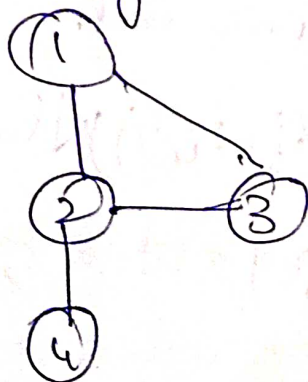
}

else if $(w \neq v)$ then $LT[u] = \min(LT[u], dfn[w])$

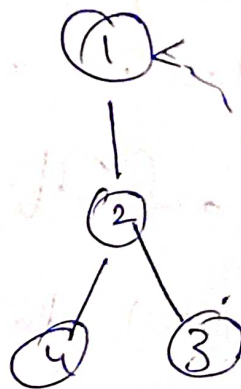
}

}

Execution: Consider the graph the we discussed in Ar(u, v) algorithm.



A graph



Spanning Tree.

(i) The method of computing dfn value and L value are same as Ar algorithm i.e

$$dfn[u] = 1 \quad L[u] = 1$$

$$dfn[u] = 2 \quad L[u] = 1$$

$$dfn[u] = 3 \quad L[u] = 1$$

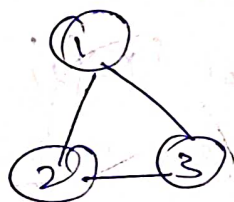
$$dfn[u] = 4 \quad L[u] = 3$$

(ii) stack is used to add the edges of adjacent vertices from the current vertex

(iii) when the condition $L[w] > dfn[u]$ is satisfied, Pop an edge from the stack until the condition satisfies, i.e.

$(x, y) = (u, w)$ or (w, u) ; where (x, y) represents an edge of stack

(iv) The two new biconnected components are



(a) component 1

(b) component 2.

Analysis:- The computing time is $O(n+e)$, where n is the number of vertices and e is the no. of edges.

Algorithm to Find Biconnected components:-

Algorithm Bicomp(u, v)

// u is a start vertex for depth first search v is its parent if any in the depth-first spanning tree. It is assumed that the global array dfn is initially zero & that the global variable num is initialized to 1.
 // n is the no. of vertices in G.

Σ $dfn[u] = num; L[u] = num; num = num + 1;$
for each vertex w adjacent from u do

Σ if $((v \neq w) \text{ and } (dfn[w] < dfn[u]))$ then
add (u, w) to the top of a stack s ;
if $(dfn[w] = 0)$ then.

Σ if $(L[w] > dfn[u])$ then.

Σ write ("new bicomponent")
repeat

Σ delete an edge from the top of stack
let this edge be (x, y) ;
write (x, y) ;

}

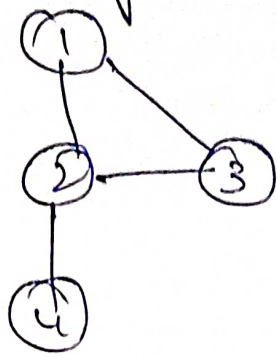
until $((x, y) = (u, w) \text{ or } ((x, y) = (w, u)))$

}
BiComp(w, u); // w is unvisited
 $L[u] = \min(L[u], L[w])$

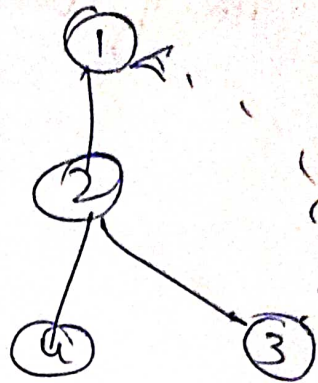
}
else if $(w \neq v)$ then $L[u] = \min(L[u], dfn[w])$

}
}

Execution:- Consider the graph the we discussed in $\Delta r(u, v)$ algorithm.



A graph



Spanning Tree.

(i) The method of computing dfn value and L value are same as Art algorithm i.e.

$$dfn[1] = 1 \quad L[1] = 1$$

$$dfn[2] = 2 \quad L[2] = 1$$

$$dfn[4] = 3 \quad L[3] = 1$$

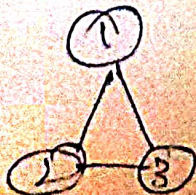
$$dfn[3] = 4 \quad L[4] = 3$$

(ii) stack is used to add the edges of adjacent vertices from the current vertex.

(iii) when the condition $L[w] > dfn[u]$ is satisfied, pop an edge from the stack until the condition satisfies. i.e.

$(x, y) = (u, w)$ or (w, u) ; where (x, y) - represent an edge of stack.

(iv) The two new bicomponent are .



(a) Component 1



(b) Component 2

Analysis:- The computing time is $O(n+e)$, where n is the no. of vertices and e is the no. of edges